

Stage 2: Progress Review

Diabetes Risk & Lifestyle Advisor
An AI-Based Decision-Support System

Course: Essentials of Artificial Intelligence

Due: Week 8 | Weight: 10%

Disclaimer: This system is a screening tool built for educational purposes only. It is NOT a substitute for professional medical diagnosis or treatment.

1. Updated Project Title and Problem Statement

Project Title: Diabetes Risk & Lifestyle Advisor - An AI-Based Decision-Support System

The Problem We're Solving:

Diabetes affects hundreds of millions of people globally, but a lot of cases go unnoticed until serious complications pop up. While early screening helps a ton, we noticed that most existing tools are pretty limited. They either just give a basic "yes/no" based on a questionnaire, or spit out a raw probability from a machine learning model. Neither of those is particularly helpful or easy to understand for an average person.

We wanted to build something better. Our system takes a more holistic approach by linking three different AI techniques together:

- First, a Neural Network crunches the clinical health metrics to estimate a baseline risk probability.
- Then, a Fuzzy Logic engine steps in to translate that raw number into a clear risk category (Low, Moderate, High, or Severe), while also factoring in the user's BMI and physical activity level.
- Finally, a Rule-Based Expert System looks at the specific risk tier and health metrics to generate personalized, easy-to-follow lifestyle and medical recommendations.

We wrapped all of this up in an interactive Streamlit web dashboard to make it as user-friendly as possible.

2. Changes Made After Proposal Feedback

We got some really helpful feedback on our initial proposal, and we've made several big changes to the project as a result:

- Connecting the AI techniques: Originally, our three AI methods felt a bit like isolated tools. We redesigned the architecture into a sequential pipeline so that each stage directly feeds its output into the next (Neural Network -> Fuzzy Logic -> Rules).
- Making rules more personal: We were told our rule-based advice felt a bit generic. We fixed this by adding metric-specific modifiers. Now, the system looks at individual metrics (like high glucose or low activity) and gives targeted advice, rather than just generic tier-based suggestions.
- Expanding the fuzzy logic: We originally only passed the NN probability to the fuzzy engine. Now, it takes in 3 variables (risk probability, BMI, and activity level) and uses 36 different rule combinations to get a much more accurate risk category.
- Better data cleaning: We realized our dataset had some biologically impossible values (like a blood pressure of 0). We implemented zero-imputation and StandardScaler normalization to clean things up.
- Adding disclaimers: To address ethical concerns, we made sure the system clearly outputs a medical disclaimer as its very first recommendation, and we documented the dataset's inherent biases.
- Building a test suite: We added a `test_pipeline.py` script to run both individual module tests and end-to-end profile tests, making sure everything is working as expected.

3. Finalized AI Techniques

Here's a closer look at the three techniques we settled on:

1. Neural Network (scikit-learn MLPClassifier)

We use this to predict the initial diabetes risk probability based on 8 clinical metrics. We built it with 2 hidden layers (16 and 8 neurons) using the ReLU activation function and Adam optimizer. To get it working well, we used an 80/20 stratified split for training and testing, and made sure to normalize the data and replace those impossible zero values with medians. It outputs a continuous probability between 0.0 and 1.0.

2. Fuzzy Logic (scikit-fuzzy)

This acts as the bridge between the complex math and human understanding. It takes the probability from the neural network, along with the patient's BMI and self-reported activity level. Using 36 fuzzy rules and trapezoidal/triangular membership functions, it calculates a risk score (0-100) and categorizes the patient into Low, Moderate, High, or Severe risk.

3. Rule-Based Expert System (Plain Python)

This is where the personalized advice comes in. Depending on the risk category, it pulls from 4 tier-specific rule sets. Then, it checks 6 different metric modifiers (like if their glucose is over 180, or if their BMI is under 18.5) to add specific recommendations. In total, it can generate about 25-30 distinct pieces of advice.

4. System Architecture

Here's a quick look at how the data flows through our application:

- 1. User Input: The patient enters 8 health metrics + their activity level on the Streamlit dashboard.
- 2. Stage 1 (Neural Network): The app feeds those 8 metrics into the trained model to get a risk probability.
- 3. Stage 2 (Fuzzy Logic): The probability, BMI, and activity level go into the fuzzy engine to determine the final risk category.
- 4. Stage 3 (Rules Engine): The risk category and specific health metrics are analyzed to generate a personalized list of recommendations.
- 5. Dashboard Output: The user sees a clean summary with color-coded risk levels, a progress bar, and their actionable recommendations.

5. Dataset Details

We're using the Pima Indians Diabetes Dataset from the UCI Machine Learning Repository. It contains 768 records with 8 numeric predictors (like Pregnancies, Glucose, Blood Pressure, BMI, Age, etc.) and a binary outcome for diabetes.

How we cleaned it: We found that several columns had '0' for values that can't actually be zero in a living person (like BMI or blood pressure). We replaced those zeros with NaN and filled them using the median value of each respective

column.

Limitations we kept in mind: The dataset only includes adult women of Pima Native American heritage from a study done in the 1960s. This means our model is quite biased towards this specific demographic and might not be entirely accurate for men, other ethnicities, or younger people.

6. AI Technique Design Details

Neural Network Setup:

- Architecture: Input (8) -> Hidden 1 (16, ReLU) -> Hidden 2 (8, ReLU) -> Output (2, Softmax)
- Optimizer: Adam (max 500 iterations)
- Performance: We're currently seeing an accuracy of about 72% (Precision: 60%, Recall: 63%, F1: 61%).

Fuzzy Logic Setup:

- Inputs: risk_probability (low, medium, high), bmi (underweight, normal, overweight, obese), and activity_level (sedentary, moderate, active).
- Output: risk_category (low, moderate, high, severe).
- Rules: 36 rules total. For example: IF risk is high AND BMI is obese AND activity is sedentary, THEN category is severe.

Rule-Based Engine Setup:

- Starts with base rules for the risk tier (e.g., Low risk gets 4 general healthy lifestyle tips).
- Checks individual metrics to add specific advice (e.g., IF blood pressure > 100, THEN recommend an antihypertensive evaluation).

7. Sample Outputs

We built a test script (test_pipeline.py) that successfully runs 10 diverse patient profiles through the entire pipeline. Our Streamlit UI displays the results cleanly, with interactive elements, progress bars, and color-coded risk alerts.

Below are screenshots of the actual system in action:

Patient Input Form:



About

Diabetes Risk & Lifestyle Advisor uses three AI techniques in a pipeline:

1.  **Neural Network** — predicts diabetes risk probability from health metrics
2.  **Fuzzy Logic** — categorizes risk considering BMI and activity level
3.  **Rule-Based Engine** — generates personalized lifestyle recommendations

Dataset Info

Pima Indians Diabetes Dataset

- 768 patients, 8 health metrics
- Source: NIDDK / UCI ML Repository
- Population: Pima Native American women, age ≥ 21



Diabetes Risk & Lifestyle Advisor

Enter your health metrics below to receive a **personalized risk assessment** and **lifestyle recommendations**.



Patient Health Metrics

Pregnancies 	Skin Thickness (mm) 	Family History Score 
1  	29.00  	0.3725  
Glucose (mg/dL) 	Insulin (μ U/mL) 	Age (years) 
120.00  	125.00  	33  
Blood Pressure (mm Hg) 	BMI (kg/m^2) 	 Activity Level 
72.00  	28.00  	 5

 Analyze Risk

 Diabetes Risk & Lifestyle Advisor — Essentials of Artificial Intelligence Group Assignment

 This tool is for educational and screening purposes only. It is NOT a substitute for professional medical advice, diagnosis, or treatment. Always consult a qualified healthcare provider.

Risk Assessment and Recommendations:

Deploy 

About

Diabetes Risk & Lifestyle Advisor uses three AI techniques in a pipeline:

1.  **Neural Network** — predicts diabetes risk probability from health metrics
2.  **Fuzzy Logic** — categorizes risk considering BMI and activity level
3.  **Rule-Based Engine** — generates personalized lifestyle recommendations

Dataset Info

Pima Indians Diabetes Dataset

- 768 patients, 8 health metrics
- Source: NIDDK / UCI ML Repository
- Population: Pima Native American women, age ≥ 21



Diabetes Risk & Lifestyle Advisor

Enter your health metrics below to receive a **personalized risk assessment** and **lifestyle recommendations**.



Patient Health Metrics

Pregnancies 	Skin Thickness (mm) 	Family History Score 
1  	29.00  	0.3725  
Glucose (mg/dL) 	Insulin (μ U/mL) 	Age (years) 
120.00  	125.00  	33  
Blood Pressure (mm Hg) 	BMI (kg/m^2) 	 Activity Level 
72.00  	28.00  	<input type="range" value="5"/>

 Analyze Risk

Risk Assessment Results

Pipeline: Neural Network $\rightarrow$ Fuzzy Logic $\rightarrow$ Rule-Based Engine

8. Problems Faced During Development

It wasn't all smooth sailing. Here are a few headaches we ran into and how we solved them:

- 1. Messy Data: The biologically impossible zeros were throwing off our neural network. Swapping them for column medians boosted our performance significantly.
- 2. Fuzzy Logic Crashes: The scikit-fuzzy library would occasionally crash if inputs hit the absolute edges (like exactly 0.0 or 1.0). We added input clamping to keep variables strictly within safe bounds.
- 3. Missing Fuzzy Rules: Sometimes a weird combination of inputs wouldn't trigger any rules, causing the app to crash. We added a fallback try/except block that defaults to mapping the raw NN probability if the fuzzy engine fails.
- 4. Generic Advice: Initially, our recommendations felt like reading a WebMD article. Adding the 6 metric-specific modifier blocks made the system feel much more like a personalized advisor.
- 5. Module State Issues: We noticed the fuzzy logic simulation was holding onto state between runs, skewing results. We fixed this by lazy-initializing the shared control system, but generating a fresh simulation instance for every new prediction.

9. Remaining Work

We're proud of our progress, but we still have a few tasks to tackle before the final submission:

- Boost Model Accuracy (High Priority): 72% is okay, but we want to try tuning the hyperparameters further or perhaps testing an ensemble method to see if we can get better results.
- Add Visualizations (Medium Priority): We plan to add Matplotlib/Plotly charts to the dashboard so users can visually see their risk distribution.
- Cross-Validation (Medium Priority): We want to replace our simple 80/20 train/test split with k-fold cross-validation so we can be more confident in our evaluation metrics.
- Polish Documentation (High Priority): Final code cleanup, adding more robust comments, and finishing up the final submission report.

10. Contributions

NOTE: Replace placeholder names with actual team member names and student IDs.

- Member 1 (Lead) [XXXXXXX]: Neural Network module, data cleaning, model training (25%)
- Member 2 [XXXXXXX]: Fuzzy Logic engine, membership functions, 36 rules (25%)
- Member 3 [XXXXXXX]: Rule-Based engine, recommendations, metric modifiers (25%)
- Member 4 [XXXXXXX]: Streamlit dashboard, UI, test pipeline, integration, report (25%)